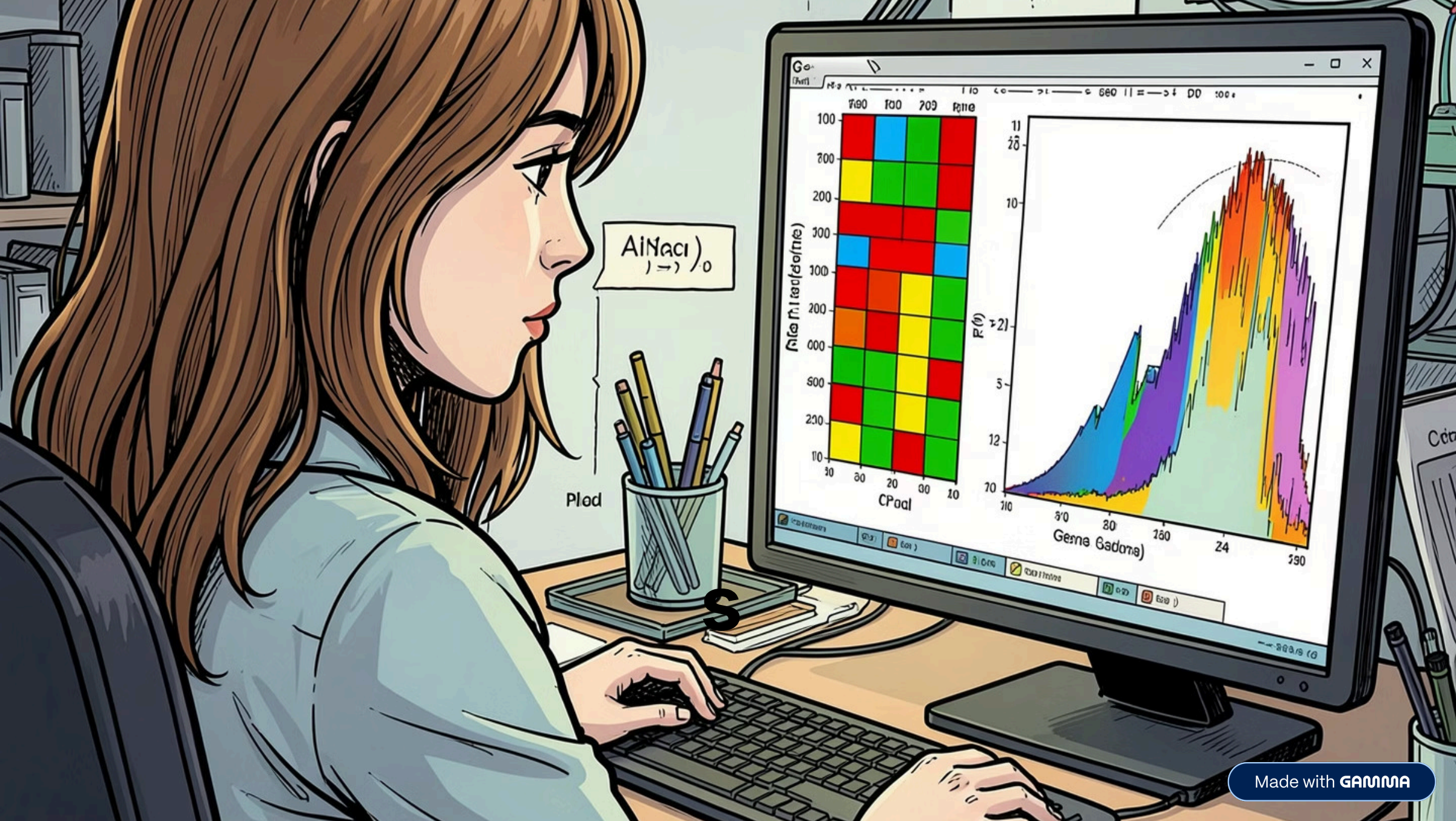


Plod

\$

Bulk RNA Sequencing Analysis

Introduction

What it is

Bulk RNA-seq is a method used to measure gene expression in a whole sample (many cells together).

It shows:

- Which genes are active
- Differences between conditions (e.g., healthy vs disease)
- How much they are expressed

What it is used for

- Comparing disease vs normal tissue
- Testing drug effects
- Studying gene function (knockout experiments)
- Comparing different tissues or stages of development

Biological basis

- DNA → RNA → protein
- RNA (mRNA) shows which genes are active
- RNA-seq measures the amount of mRNA in a sample

👉 More RNA = higher gene activity

Steps

1. Collect sample (cells/tissue)
2. Extract RNA
3. Check RNA quality
4. Convert RNA → cDNA
5. Add sequencing adapters (library prep)
6. Sequence (machine reads fragments)
7. Map reads to genome
8. Count reads per gene
9. Compare groups (find differentially expressed genes)

Step1: Quality Control using FastQC

Tool Used

FastQC

Purpose of FastQC

FastQC checks the quality of raw sequencing reads before downstream analysis.

It helps answer:

- Are sequencing reads high quality?
- Is adapter contamination present?
- Is GC content normal?
- Are reads duplicated excessively?
- Is trimming required?

Input Files

FastQC takes raw sequencing FASTQ files as input.

Example:

sample_R1.fastq

sample_R2.fastq

- R1 = forward reads
- R2 = reverse reads

Each FASTQ file is analyzed separately.

Main FastQC Command

Example:

```
fastqc sample_R1.fastq -o fastqc_results/
```

Important Parameter Used

Output Generated

For each FASTQ file, FastQC generates:

A. HTML Report

Visual quality report containing:

- quality graphs
- adapter contamination plots
- GC content plots

Main file used for QC interpretation.

B. ZIP File

Contains:

- raw QC statistics
- detailed FastQC data

Important Quality Metrics Checked by FastQC

A. Per Base Sequence Quality

Most important metric.

Shows sequencing quality across read length.

Good quality:

- high Phred scores throughout reads

Poor quality at read ends:

- indicates trimming may be needed

B. Adapter Content

Checks whether adapter sequences are present in reads.

High adapter contamination suggests:

- adapter trimming required

Usually removed later using:

Trimmomatic

C. GC Content

Measures GC percentage distribution.

Abnormal GC content may indicate:

- contamination
- sequencing bias
- library preparation issues

D. Sequence Duplication Levels

Checks repeated reads.

High duplication may indicate:

- PCR bias
- low library complexity

E. Overrepresented Sequences

Detects highly repeated sequences.

Possible causes:

- adapters
- contamination
- ribosomal RNA contamination

Why This Step Is Important

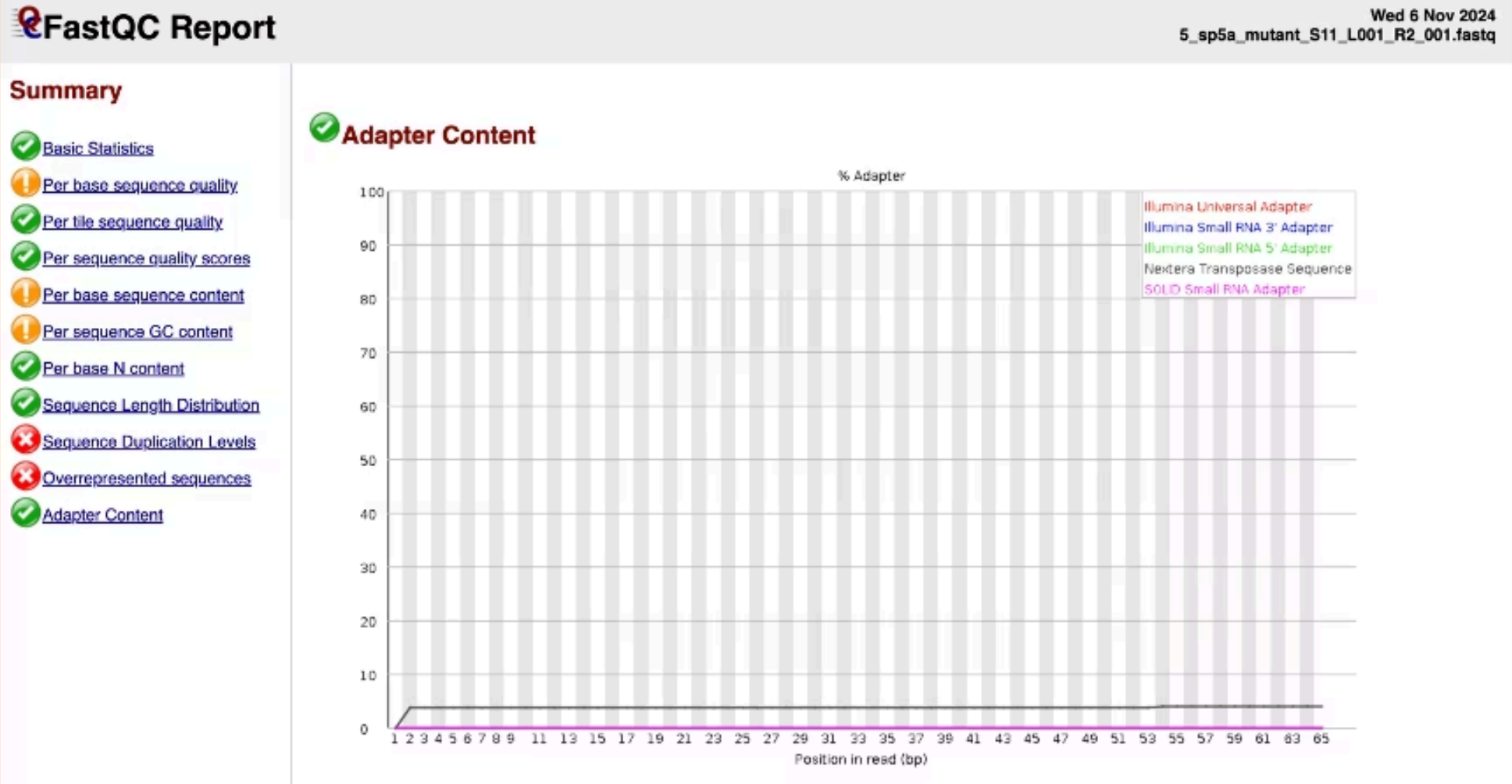
FastQC ensures sequencing data quality before analysis.

Without QC:

- poor-quality reads may align incorrectly
- contamination may affect results
- sequencing bias may produce unreliable gene expression results

This step helps determine:

- whether trimming is needed
- whether sequencing quality is acceptable



BEFORE ADAPTER TRIMMING

Step 2. Preprocessing Using Trimmomatic

Introduction

In bulk RNA-seq analysis, raw sequencing reads obtained from the sequencing machine contain unwanted sequences and low-quality regions. Before alignment and downstream analysis, the reads must be cleaned through a preprocessing step called read trimming.

In this pipeline, the software used for preprocessing is Trimmomatic.

The purpose of this step is to:

- remove adapter contamination
- remove low-quality bases
- remove poor-quality read regions
- remove very short reads

This improves:

- alignment quality
- gene quantification accuracy
- differential expression analysis reliability

Input Files

Each sample contains two FASTQ files:paired end sequencing files R1 AND R2

R1.fastq

R2.fastq

What Are Adapter Sequences?

During RNA-seq library preparation, artificial DNA sequences called adapters are attached to RNA fragments.

These adapters are required for sequencing.

Example:

Adapter + RNA fragment + Adapter

Sometimes the RNA fragment is shorter than the sequencing read length. In such cases, the sequencing machine reads into the adapter sequence.Sequencing company gives info about adpters they used for each sample.

Example:

ACTGTTGCAAGATCGGAAG

Here:

- ACTGTTGCA = real RNA sequence
- AGATCGGAAG = adapter contamination

Adapter contamination causes:

- poor alignment
- false mapping
- inaccurate gene counts

Therefore adapter trimming is necessary.

Trimmomatic compares each read against these adapter sequences and removes matching regions.

Trimmomatic Command Used

java -jar trimmomatic.jar PE -threads 2 \

ILLUMINACLIP:"\$ADAPTER_FILE":2:30:10 \

LEADING:3 \

TRAILING:3 \

SLIDINGWINDOW:4:15 \

MINLEN:50

Explanation of Parameters

1. PE (Paired-End Mode)

PE

Meaning:

- paired-end sequencing mode is used
- both R1 and R2 reads are processed together

This preserves read pairing information.

2. Threads

-threads 2

Meaning:

- uses 2 CPU cores

Purpose:

- faster processing using parallel computation

3. Adapter Trimming

ILLUMINACLIP:"\$ADAPTER_FILE":2:30:10

Purpose:

- remove adapter contamination from reads

Parameter Meaning

a) 2

Seed mismatches = 2

Meaning:

- adapter matching allows up to 2 mismatches

Reason:

- sequencing errors may occur
- slight mismatches should still be recognized as adapters

This is a commonly used standard setting.

b) 30

Palindrome clip threshold = 30

Used mainly for paired-end adapter detection.

Purpose:

- detects read-through adapters in paired-end sequencing

Higher value:

- stricter matching

This is a commonly used standard parameter.

c) 10

Simple clip threshold = 10

Purpose:

- controls normal adapter matching sensitivity

Lower value:

- more aggressive trimming

This value provides balanced adapter trimming.

4. LEADING:3

LEADING:3

Purpose:

- remove low-quality bases from the beginning of reads

Meaning:

- bases with quality score below 3 are removed

Reason:

- sequencing quality can sometimes be poor at read starts

This is a mild and commonly used filtering threshold.

5. TRAILING:3

TRAILING:3

Purpose:

- remove low-quality bases from the end of reads

Meaning:

- remove bases with quality score below 3 at the read tail

Reason:

- sequencing quality often decreases toward the end of reads

This improves alignment reliability.

6. SLIDINGWINDOW:4:15

SLIDINGWINDOW:4:15

This is one of the most important quality filtering steps.

Parameter Meaning

Window size = 4

Trimmomatic checks:

- 4 bases at a time

Quality threshold = 15

If average quality of the 4-base window becomes less than 15:

- the remaining part of the read is trimmerimme

Why Is This Important?

Sequencing quality usually drops toward the end of reads.

Low-quality bases may:

- cause alignment errors
- increase false positives
- reduce mapping accuracy

This step removes poor-quality regions while preserving good-quality sequence.

7. MINLEN:50

MINLEN:50

Purpose:

- discard reads shorter than 50 bp after trimming

Reason:

- very short reads map ambiguously
- short reads may align to multiple genomic locations

This improves mapping specificity and reduces noise.

50 bp is a commonly used cutoff for bulk RNA-seq analysis.

Output Files Generated

After trimming, four output FASTQ files are generated.paired and unpaired for each read

Paired Reads

3_wt_R1_paired.fastq

3_wt_R2_paired.fastq

These contain:

- high-quality paired reads- use for further analysis steps
- both read mates survived filtering

Unpaired Reads

3_wt_R1_unpaired.fastq

3_wt_R2_unpaired.fastq

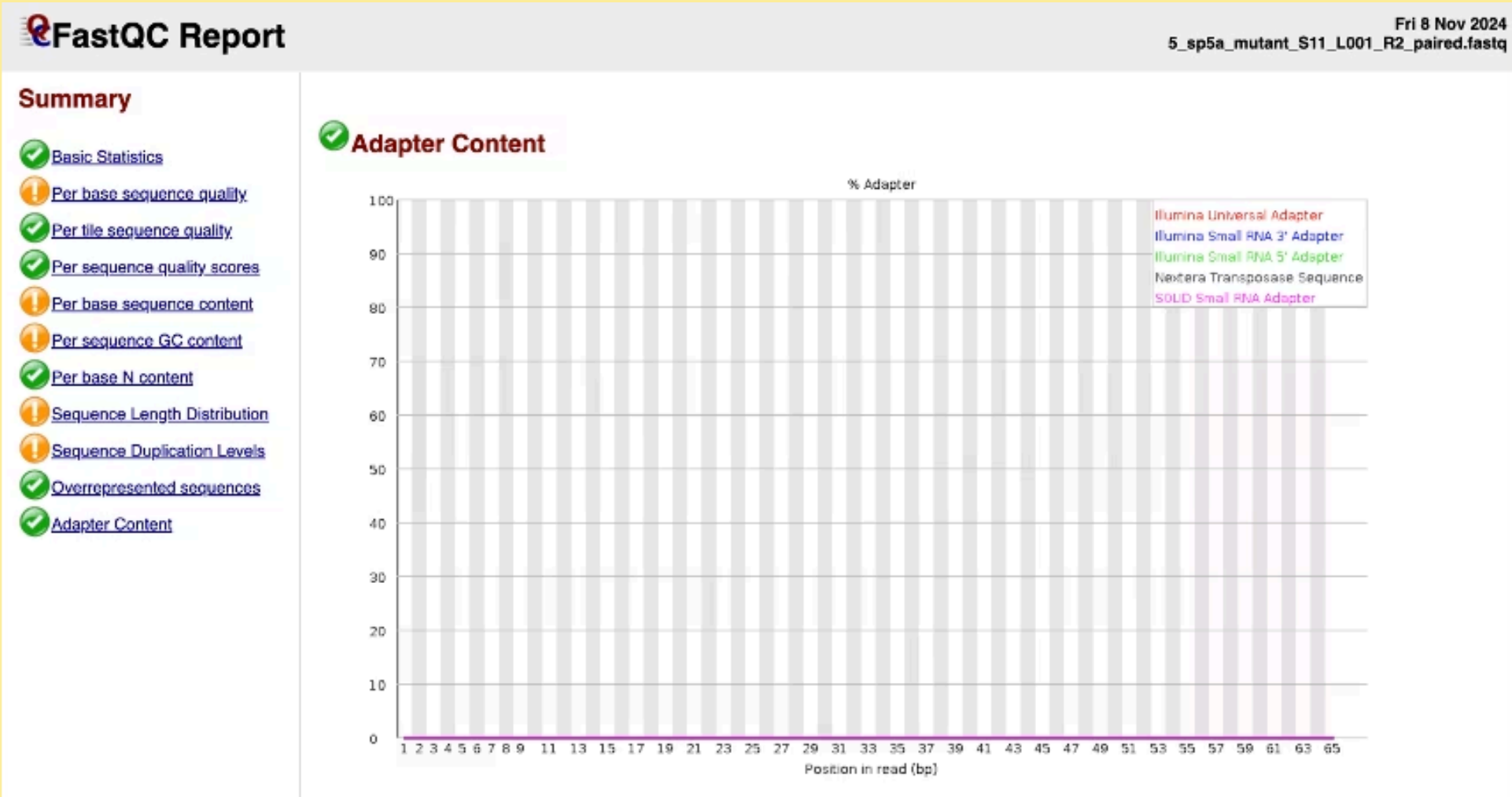
These contain:

- reads where one mate passed filtering
- the other mate failed filtering

Some pipelines discard these reads, while others align them separately.

Step 3: After Trim again do QC

To makesure trimming works properly



Step 4: STAR Genome Indexing + RNA-seq Alignment

Tool Used

STAR

STAR is a splice-aware RNA-seq aligner used for:

- building genome index (preparation step)
- aligning RNA-seq reads to a reference genome
- generating BAM files for downstream analysis

There are two main functions , Genome indexing and alignment.

PART A: STAR Genome Indexing

Purpose of Indexing

STAR indexing creates a searchable genome database so that:

- reads can be aligned quickly
- splice junctions are pre-annotated
- alignment becomes accurate and fast

It is like building a “reference genome dictionary”

There are 2 main files needed.

Inputs

- **Reference genome FASTA file**
- **Gene annotation file (GTF)**

Output

- STAR genome index folder

Contains:

- genome reference structure
- splice junction database
- mapping index files

Important Parameters

--runMode genomeGenerate

- tells STAR to build genome index

--runThreadN 16

- uses 16 CPU cores
- speeds up indexing

--genomeDir

- output folder for genome index

--genomeFastaFiles

- reference genome sequence (FASTA)

--sjdbGTFfile

- gene annotation file (GTF)
- provides exon/intron structure

--sjdbOverhang 149

- calculated as: read length – 1 (150 – 1 = 149)
- improves splice junction detection accuracy

Importance of Indexing

Without indexing:

- STAR cannot align reads

With indexing:

- faster alignment
- better splice detection
- accurate gene mapping

PART B: STAR Alignment

Purpose of Alignment

STAR maps RNA-seq reads to the reference genome to:

- determine genomic location of reads
- detect exon–exon splice junctions
- generate BAM files for downstream analysis

Inputs

- Paired-end FASTQ files (R1 and R2)
- STAR genome index (from previous step)

Output

- Sorted BAM file (main output)
- Alignment log files

BAM file contains:

- genomic position of each read
- alignment quality information

Important Parameters

--genomeDir

- path to STAR genome index

--readFilesIn

- input FASTQ files (R1 and R2)

--runThreadN 16

- uses 16 CPU cores for faster alignment

--outFileNamePrefix

- prefix for output files per sample

--outSAMtype BAM SortedByCoordinate

- outputs BAM file
- already sorted by genomic position
- required for downstream tools

How STAR Alignment Works

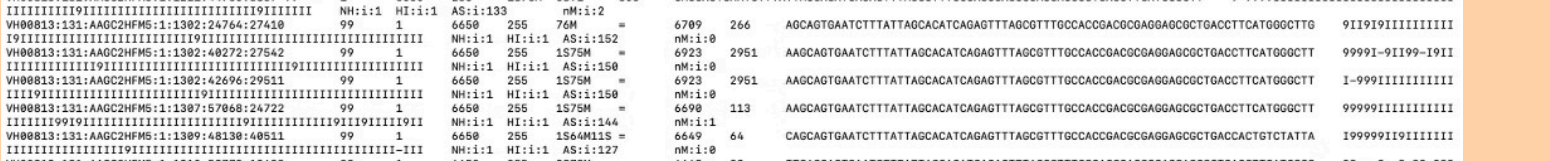
For each read:

1. Reads FASTQ sequence
2. Matches against genome index
3. Finds best genomic location
4. Allows spliced alignment (exon–intron structure)
5. Writes result into BAM file

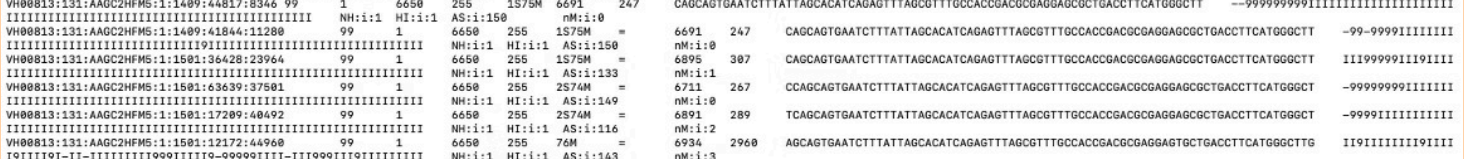
Importance of STAR Alignment

This step is essential because it:

- converts raw reads into genomic coordinates
- enables gene-level counting
- captures splicing events
- provides data for expression analysis



ALIGNMENT - visual exploration of aligned genomic data (BAM FILE - INDEXING) using IGV



BAM FILE

Step5: Read Counting using HTSeq-count (after BAM generation)

Tool Used: HTSeq-count

Purpose of HTSeq-count

HTSeq-count assigns aligned reads from BAM files to genes using a reference annotation file (GTF).

Output:

- number of reads mapped to each gene
- gene expression counts per sample

Basic Idea

For each aligned read:

- Read genomic position from BAM file
- Compare with gene annotation in GTF
- If overlap exists:
 - assign read to gene
- Otherwise classify as:
 - intergenic
 - ambiguous
 - low quality

Input Files

HTSeq-count mainly requires:

A. BAM File

Generated after alignment using:

- STAR

Contains:

- aligned sequencing reads

Important:

- BAM must be sorted by genomic position

B. GTF File

Annotation file containing:

- genes
- exons
- transcripts

Example:

gene location

exon coordinates

gene IDs

Important:

- GTF must match genome version used during alignment

Example:

- GRCz11 genome → use GRCz11 GTF

Main HTSeq-count Command

Example:

htseq-count -f bam -r pos -s no -i gene_id sample.bam annotation.gtf

This is the main read-counting step.

Parameters Used and Their Meaning

A. -f bam

-f bam

Meaning:

- input file format is BAM

Purpose:

- tells HTSeq to read aligned BAM files

B. -r pos

-r pos

Meaning:

- BAM file is sorted by genomic position

Important because:

- HTSeq reads alignments sequentially
- unsorted BAM may cause errors

C. -s no

-s no

One of the most important parameters.

Meaning:

- RNA-seq library is unstranded

HTSeq will count reads regardless of strand direction.

What is strandedness?

RNA can be sequenced as:

- stranded
- unstranded

Unstranded (-s no)

Reads can map to either strand.

Common in many standard RNA-seq datasets.

Why is this important?

Wrong strand setting causes:

- incorrect gene counts
- loss of reads
- inaccurate differential expression results

Therefore strandedness must match library preparation method.

D. -i gene_id

-i gene_id

Meaning:

- use gene_id field from GTF annotation

Purpose:

- counts are grouped by gene ID

Example output:

ENSG000001234

ENSG000005678

Important Summary Categories

HTSeq outputs extra lines such as:

__no_feature

__ambiguous

__too_low_aQual

Meaning of These Categories

__no_feature

Reads that do not overlap annotated genes.

Possible reasons:

- intergenic regions
- incomplete annotation

__ambiguous

Reads overlap multiple genes.

HTSeq cannot confidently assign them.

__too_low_aQual

Reads with low alignment quality.

Usually filtered out.

Output Generated

HTSeq-count produces a count file for each sample.

Example:

geneA 120

geneB 55

geneC 0

Meaning:

- GeneA has 120 reads
- GeneB has 55 reads
- GeneC has 0 reads

These values represent gene expression levels.

Building Final Count Matrix

Counts from all samples are combined into one matrix.

Example:

Gene_ID Sample1 Sample2 Sample3

geneA 120 98 110

geneB 55 60 52

geneC 0 4 2

Structure of Count Matrix

- Rows = genes
- Columns = samples
- Values = read counts

This matrix is the main input for differential expression analysis.

normalized_counts_new							
	1_sp5a_mutant	1_wt	3_wt	4_sp5a_mutant	4_wt	5_sp5a_mutant	
ENSDARG000000000001	147.162719616182	137.144630960224	119.838313857203	154.240702958696	96.9756490181959	141.910803371071	
ENSDARG000000000002	493.048819735967	434.139957609185	397.282167756909	362.521944180292	405.015945899524	405.638845034124	
ENSDARG000000000018	706.810726331734	1043.5707349225	964.516853347669	782.461960264916	945.037207098889	761.043777370525	
ENSDARG000000000019	1742.32066582078	2748.34207473932	2308.15855417085	1665.12408522563	2402.52416734295	1686.6036188261	
ENSDARG000000000068	583.279976288954	410.52565029153	440.133443621	442.456907027499	374.592212874208	494.804040072585	
ENSDARG000000000069	1119.29601343111	577.642286693393	678.357485712894	1085.31414344659	594.21353565071	1007.18994958937	
ENSDARG000000000086	794.893522014412	1893.68579835806	1975.51644661571	1065.048941598	1899.58183076819	865.279146218303	
ENSDARG000000000103	4543.78324070401	5327.75102789849	5010.69410485359	3914.56149041887	4873.50173399286	4136.7627106576	
ENSDARG000000000142	33.299593489793	190.730943719516	192.46759498278	74.3057401114884	183.493139808939	52.7456083326106	
ENSDARG000000000151	225.577891382468	208.895795502327	154.70036879748	156.492392052983	193.951298036392	178.330390076922	
ENSDARG000000000161	711.107448072353	593.990653297922	578.129077759597	571.929029949032	506.745303202925	630.435604356441	
ENSDARG000000000175	153.607802227109	60.8522534724171	108.943921688366	173.38006026014	127.399382043512	133.11986864897	
ENSDARG000000000183	1027.99067644296	1477.71069253168	1375.59858451844	1181.01092995381	1363.36353619699	1329.94283867225	
ENSDARG000000000189	1058.06772862729	1659.35921035979	1687.17820054716	1239.55484640528	1677.10828302056	1313.61681704549	
ENSDARG000000000212	22785.5153904996	4563.91901043128	5015.05186172113	22396.4255763309	4014.98201768472	19331.2654539018	
ENSDARG000000000229	750.852124173073	670.283030785729	488.795081975136	467.225487064662	526.710878000789	511.130061699346	
ENSDARG000000000241	790.5986802273794	946.388777884457	995.747444231667	795.972094830641	1016.34283137697	769.834712092626	
ENSDARG000000000324	429.672174061845	668.46645607448	553.4351221769	395.171436047461	574.247960852846	401.871301581795	

Step 6- Differential analysis ANALYSIS and other visualization steps.

In bulk RNA-seq, Differential Gene Expression analysis means:

Finding genes whose expression levels are significantly different between two or more biological conditions (e.g., mutant vs wild-type, treated vs control).

Output of DGE analysis

For every gene, you get:

- log2 Fold Change (log2FC)
→ how much expression changes
- p-value
→ statistical significance
- adjusted p-value (padj / FDR)
→ corrected for multiple testing

What tools are used in R for bulk RNA-seq DGE?

The main tools are:

- DESeq2 (MOST POPULAR)

DESeq2

Used for:

- raw count data
- biological replicates
- robust normalization + statistics

Features:

- built-in normalization
- negative binomial model
- shrinkage of variance
- very standard in publications

Data Normalization is automatically done in the Deseq 2 object creation step.

Analysis flow

- Read counts
- Check sample order
- Set WT as reference
- Run DESeq2
- Extract results
- Annotate genes
- Filter significant genes

ENSEMBL_ID	baseMean	log2FoldChange	lfcSE	pvalue	padj	SYMBOL	GENENAME	GO	EVIDENCE	ONTOLOGY
ENSDARG00000000142	121.173770074188	1.75613686956773	0.280047031078857	4.5835860731935E-11	4.15888926618337E-10	tmem267	transmembrane protein 267	GO:0003674	ND	MF
ENSDARG00000000142	121.173770074188	1.75613686956773	0.280047031078857	4.5835860731935E-11	4.15888926618337E-10	tmem267	transmembrane protein 267	GO:0008150	ND	BP
ENSDARG00000000142	121.173770074188	1.75613686956773	0.280047031078857	4.5835860731935E-11	4.15888926618337E-10	tmem267	transmembrane protein 267	GO:0016020	IEA	CC
ENSDARG00000000142	121.173770074188	1.75613686956773	0.280047031078857	4.5835860731935E-11	4.15888926618337E-10	tmem267	transmembrane protein 267	GO:0016021	IEA	CC
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0005886	IBA	CC
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015175	IBA	MF
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015180	IBA	MF
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015183	IBA	MF
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015194	IBA	MF
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015195	IBA	MF
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015293	IEA	MF
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015813	IBA	BP
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0015825	IBA	BP
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0016020	IEA	CC
ENSDARG000000000551	1736.14873429589	2.11770250013894	0.11677506291532	1.85597091621273E-74	5.53195331213657E-71	slc1a4	solute carrier family 1 member 4	GO:0016021	IEA	CC
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0005313	IBA	MF
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0005326	IBA	MF
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0006811	IEA	BP
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0006814	IEA	BP
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0006820	IBA	BP
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0006836	IEA	BP
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0015293	IEA	MF
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0016020	IEA	CC
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0016021	IBA	CC
ENSDARG000000001127	772.031891693075	1.83011690705005	0.239422758254542	2.51855383985707E-15	4.40285310200819E-14	slc17a6a	solute carrier family 17 member 6a	GO:0016021	IEA	CC

SPECIAL POINTS

1. Column Name & Sample Order Check

✔ Purpose:

Ensures that sample metadata matches count matrix columns exactly

✔ Why important:

- prevents wrong sample assignment
- avoids incorrect differential expression results

✔ Code check:

all(colnames(countdata) == rownames(sample_info))

✔ Fix if mismatch:

sample_info <- sample_info[colnames(countdata), , drop = FALSE]

2. REFERENCE LEVEL SETTING (IMPORTANT CONCEPT)

Meaning:

Reference level = baseline group used for comparison

Example:

- WT = reference → mutant vs WT
- control = reference → treated vs control

It determines:

direction of log2 fold change

1. DATA-LEVEL REFERENCE SETTING (MOST COMMON)

This is done before running DESeq2

✔ Option 1: relevel() (BEST PRACTICE)

sample_info\$condition <- relevel(sample_info\$condition, ref = "wt")

✔ What it does:

- explicitly sets WT as baseline
- changes reference level permanently in dataset

✔ Use case:

- recommended standard method
- clear and reproducible

✔ Option 2: factor(levels=...)

sample_info\$condition <- factor(sample_info\$condition, levels = c("wt", "mutant"))

✔ What it does:

- first level becomes reference automatically

✔ Important rule:

- first level = reference

2. RESULTS-LEVEL REFERENCE (CONTRAST METHOD)

This is done after DESeq2 runs

✔ Method: results(... contrast=)

res <- results(dds, contrast = c("condition", "mutant", "wt"))

✔ What it does:

- ignores factor reference level
- directly defines comparison

✔ Output: $\log_2(\textit{mutant}/\textit{wt})$

Key feature:

- independent of data-level reference
- gives direct control of comparison

3. Gene Description / Annotation Addition

✔ Purpose:

Adds biological meaning to gene IDs

✔ Example code:

res\$symbol <- mapIds(org.Dr.eg.db, keys = rownames(res), column = "SYMBOL", keytype = "ENSEMBL", multiVals = "first")

res\$gene_name <- mapIds(org.Dr.eg.db, keys = rownames(res), column = "GENENAME", keytype = "ENSEMBL", multiVals = "first")

CODE BLOCKS FOR EACH TASK IN DEGS

Load libraries

```
library(DESeq2)

library(ggplot2)

library(pheatmap)

library(org.Dr.eg.db)

library(AnnotationDbi)

library(clusterProfiler)

library(ggrepel)
```

Read count matrix

```
countdata <- read.table("counts_matrix.txt",

header = TRUE,

row.names = 1,

sep = "\t",

check.names = FALSE)

Sample metadata

sample_info <- data.frame(

row.names = colnames(countdata),

condition = c("mutant","wt","mutant","wt","mutant","wt")

)
```

Check sample order

```
all(colnames(countdata) == rownames(sample_info))

sample_info <- sample_info[colnames(countdata), , drop = FALSE]

Set reference (WT baseline)

sample_info$condition <- factor(sample_info$condition)

sample_info$condition <- relevel(sample_info$condition, ref = "wt")

Build DESeq2 object

dds <- DESeqDataSetFromMatrix(

countData = countdata,

colData = sample_info,

design = ~ condition

)
```

Filter low counts

```
dds <- dds[rowSums(counts(dds)) > 10, ]

Run DESeq2

dds <- DESeq(dds)

res <- results(dds)

resOrdered <- res[order(res$padj), ]
```

Annotation

```
resOrdered$symbol <- mapIds(org.Dr.eg.db,

keys = rownames(resOrdered),

column = "SYMBOL",

keytype = "ENSEMBL",

multiVals = "first")

resOrdered$gene_name <- mapIds(org.Dr.eg.db,

keys = rownames(resOrdered),

column = "GENENAME",

keytype = "ENSEMBL",

multiVals = "first")

resOrdered$entrez <- mapIds(org.Dr.eg.db,

keys = rownames(resOrdered),

column = "ENTREZID",

keytype = "ENSEMBL",

multiVals = "first")
```

Significant genes

```
sig_res <- subset(resOrdered,

padj < 0.05 &

abs(log2FoldChange) > 1)
```

Save outputs

```
write.csv(as.data.frame(resOrdered), "all_results.csv")

write.csv(as.data.frame(sig_res), "sig_genes.csv")
```


PLOTS

MA Plot

Purpose:

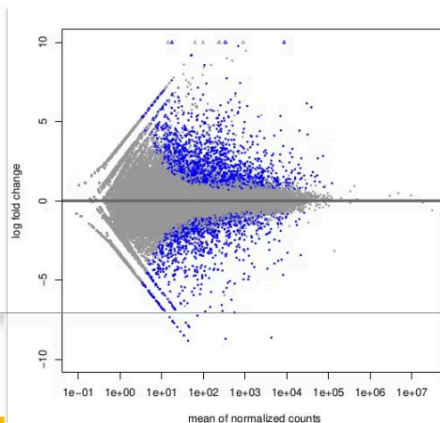
- Shows log fold change vs expression level
- Detects bias in differential expression
- Helps check if normalization worked well

code:

```
plotMA(res, ylim = c(-5, 5))
```

✓ Simple QC plot (standard in DESeq2)

MA plot Explained!



PCA Plot

Purpose:

- Shows sample similarity
- Checks if replicates cluster together
- Detects batch effects or outliers

PCA Plot Meaning

In a PCA plot:

- Each dot = one sample
- Distance between dots = similarity/difference
- Axes are PC1 and PC2

PC1 → major variation
PC2 ↑ secondary variation

If two groups separate clearly:

- Their data patterns are different

PCA:

```
vsd <- vst(dds, blind = FALSE)

pcaData <- plotPCA(vsd, intgroup = "condition", returnData = TRUE)

percentVar <- round(100 * attr(pcaData, "percentVar"))

ggplot(pcaData, aes(PC1, PC2, color = condition)) +

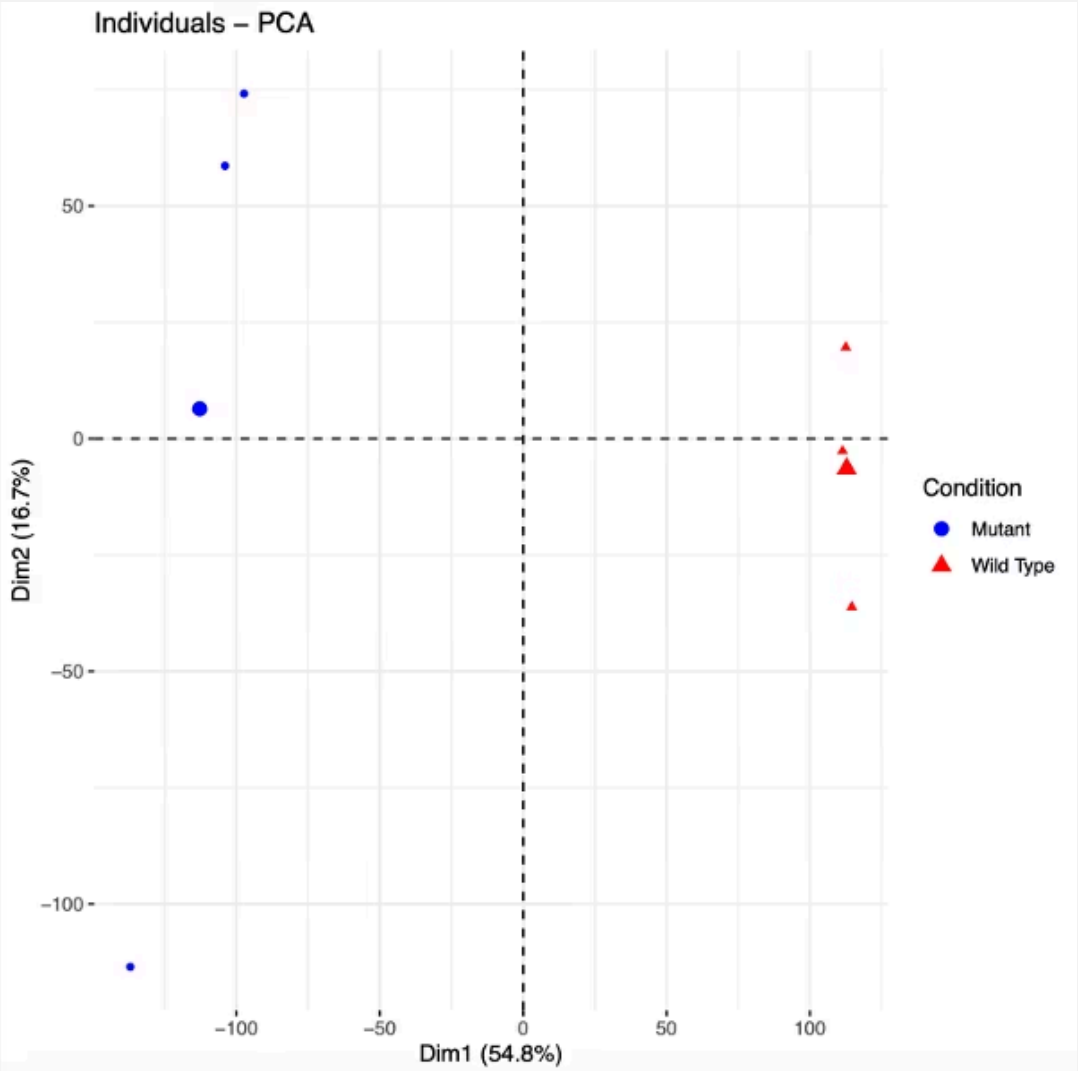
geom_point(size = 4) +

theme_minimal() +

labs(title = "PCA Plot",

x = paste0("PC1: ", percentVar[1], "%"),

y = paste0("PC2: ", percentVar[2], "%"))
```



Volcano Plot

Purpose:

- Shows significance vs fold change
- Quickly identifies key genes
- Highlights up/down regulated genes

volcano:

```
res_df <- as.data.frame(resOrdered)

res_df$group <- "Not Significant"

res_df$group[res_df$padj < 0.05 & res_df$log2FoldChange > 1] <- "Up"

res_df$group[res_df$padj < 0.05 & res_df$log2FoldChange < -1] <- "Down"

ggplot(res_df, aes(x = log2FoldChange,

y = -log10(padj),

color = group)) +

geom_point(alpha = 0.7) +

theme_minimal() +

scale_color_manual(values = c("blue","grey","red")) +

geom_vline(xintercept = c(-1,1), linetype = "dashed") +

geom_hline(yintercept = -log10(0.05), linetype = "dashed") +

labs(title = "Volcano Plot")
```

Top gene labeling volcano

```
top <- head(res_df[order(res_df$padj), ], 10)

ggplot(res_df, aes(log2FoldChange, -log10(padj))) +

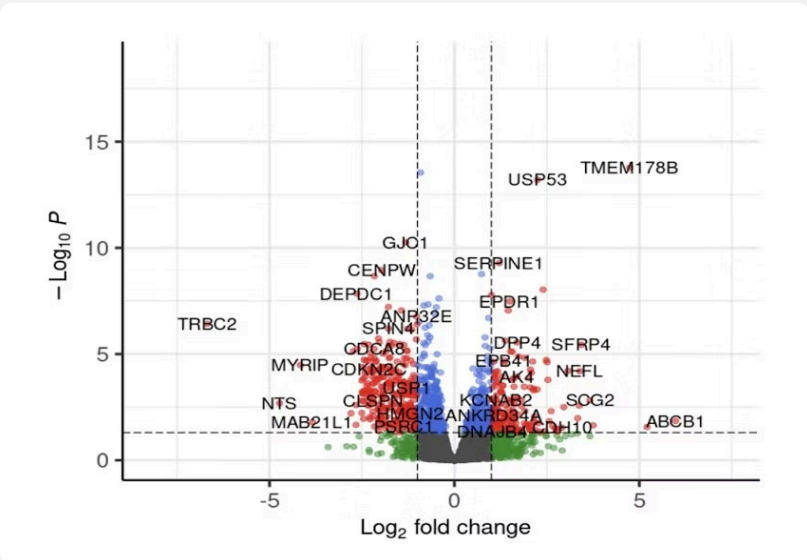
geom_point(alpha = 0.6) +

geom_text_repel(data = top,

aes(label = symbol),

size = 3) +

theme_minimal()
```



Heatmap (Top genes)

Purpose:

- Shows expression patterns of DE genes
- Visual clustering of genes & samples

```
top_genes <- head(rownames(sig_res), 50)
```

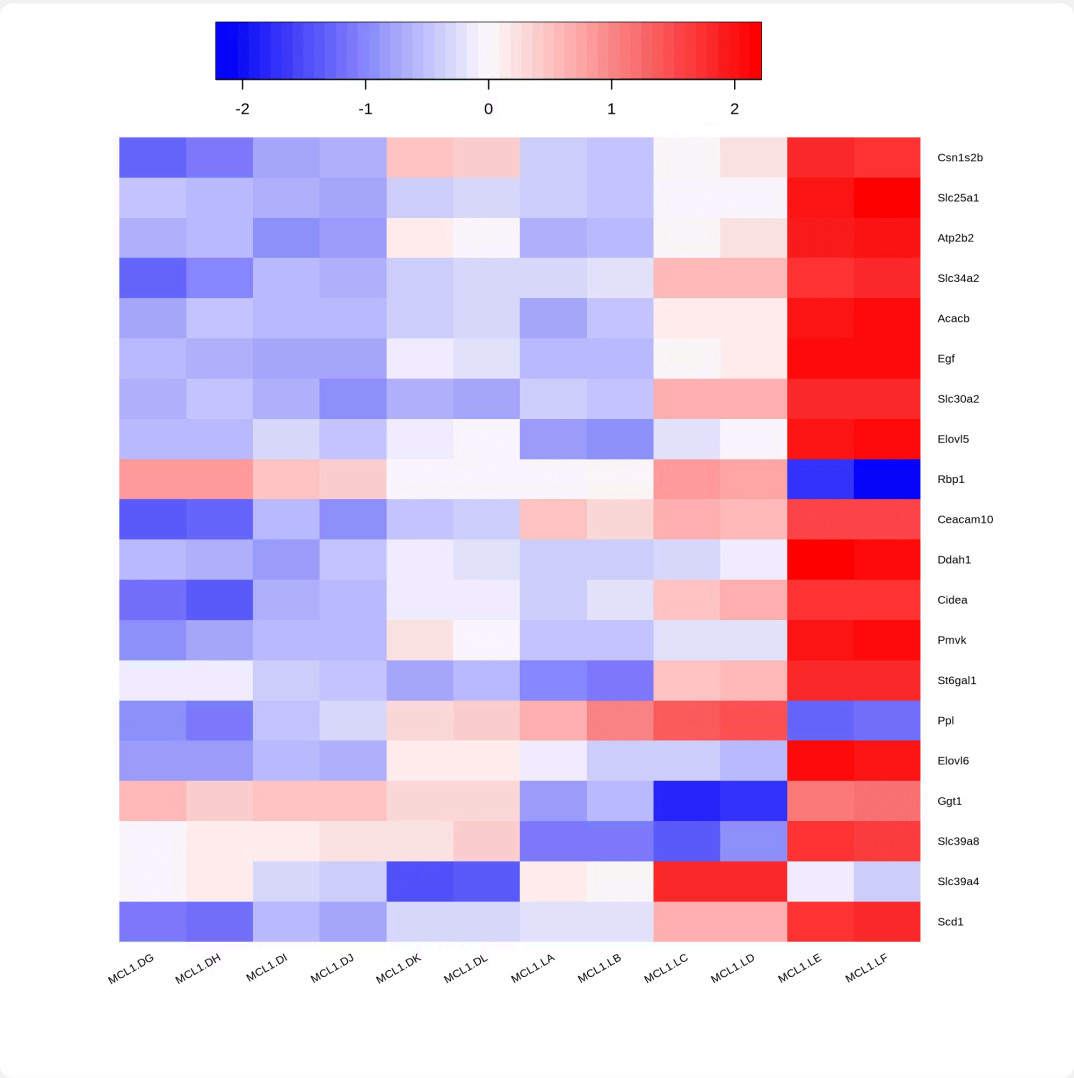
```
mat <- assay(vsd)[top_genes, ]
```

```
pheatmap(mat,
```

```
scale = "row",
```

```
annotation_col = sample_info,
```

```
main = "Top 50 DE Genes")
```



GO Analysis

Purpose:

- Converts gene list → biological meaning
- Finds enriched biological processes

```
gene_list <- na.omit(sig_res$entrez)
```

```
ego <- enrichGO(
```

```
gene = gene_list,
```

```
OrgDb = org.Dr.eb.db,
```

```
keyType = "ENTREZID",
```

```
ont = "BP",
```

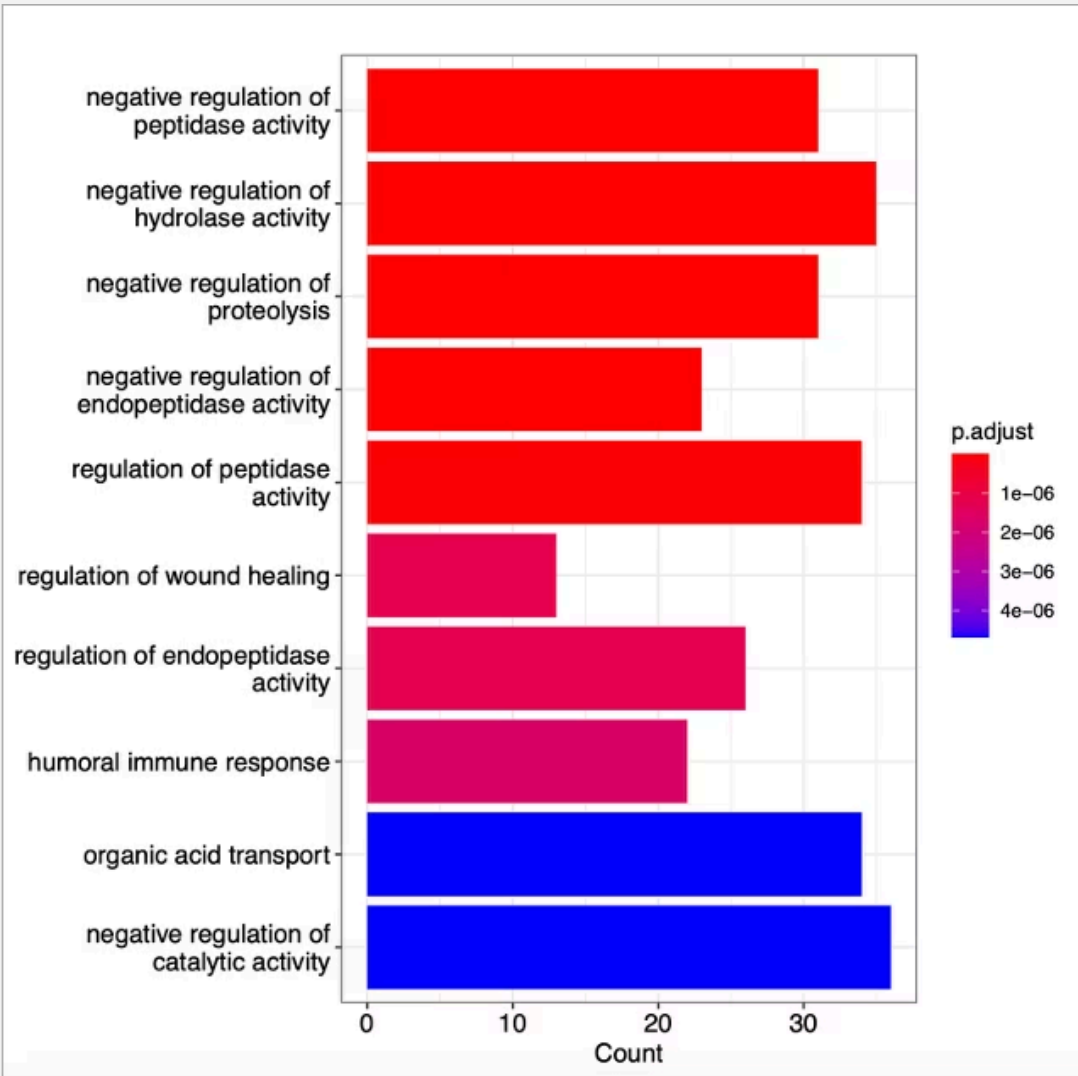
```
readable = TRUE
```

```
)
```

GO plots:

```
barplot(ego, showCategory = 10)
```

```
dotplot(ego, showCategory = 10)
```



What is log fold change shrinkage?

In bulk RNA-seq analysis (especially with DESeq2), log fold change shrinkage is a method used to:

Make gene expression changes (log2 fold changes) more stable and less noisy, especially for low-count genes.

Problem without shrinkage

Raw log2FC estimates can be:

- very large for low-count genes
- unstable across replicates
- noisy due to sampling variation

Example issue:

A gene with very low counts:

Sample	Counts
WT	1
Mutant	10

→ looks like huge change (log2FC ≈ 3.3), but this is not reliable

What shrinkage does

Log fold change shrinkage:

Pulls extreme log2FC values toward zero when data is uncertain.

So:

- large but uncertain changes → reduced
- stable strong changes → mostly unchanged

 4. Before vs after shrinkage

Gene type	Raw log2FC	Shrunk log2FC
high count, strong signal	5.0	4.8
low count, noisy	6.0	1.2
stable gene	1.5	1.4

How to do it in DESeq2

Method used:

```
resLFC <- lfcShrink(dds,  
  
coef = "condition_mutant_vs_wt",  
  
type = "apeglm")
```

Types of shrinkage methods

Method	Description
apeglm	BEST (modern, accurate)
ashr	alternative Bayesian method
normal	older method (less recommended)

When to use shrinkage

- ✓ Volcano plots
- ✓ Ranking genes
- ✓ Biological interpretation
- ✗ Not needed for raw statistical testing